

Chapter 1. From LLMs to Agents: The Foundational Blueprint

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mcronin@oreilly.com.

One of the defining traits of human intelligence is the way we combine inner reasoning with concrete actions, and this maps surprisingly well to how large language model (LLM) agents operate when paired with tools. Take the process of building a coding project. As a human developer you begin with a prompt, the client’s request. First comes reasoning, where you sketch out a plan of how to approach it. Then comes action, such as searching for documentation, writing functions, or debugging errors. Feedback enters the loop when tests fail, a peer review points out gaps, or the client tests the application. Each step is not static but iterative, with reasoning adapting to new insights and actions changing in response.

The same applies to a single LLM agent. Given a task, the agent first reasons about what is missing or what step comes next, then takes actions by calling tools to retrieve data, run code, or check results. Like a developer’s feedback loop, the environment provides signals such as errors, gaps, or confirmations that guide the next iteration. External support, such as code libraries or documentation, maps to tools like retrieval systems or web searches, which refine the output beyond the model’s own raw ability.

An LLM agent is a large language model embedded in a loop of reasoning, acting, and feedback, where it can call external tools and adapt its behavior based on results. Unlike a standalone LLM, which is limited to static text generation, an agent operates as a decision-making entity within a workflow. In this book, I will use AI agents, agents, and LLM agents interchangeably. Strictly speaking, an AI agent or agent does not need to in-

volve a language model at all. For example, in pure reinforcement learning agents are trained directly through trial and error interaction with an environment. The benefit of using an LLM is that it brings strong generalization, reasoning, and natural language capabilities, which makes agent behavior more flexible and broadly applicable beyond narrowly defined environments.

Just as individual skills eventually meet their limits, a single agent can only take a workflow so far before complexity demands coordination. If you think about it, in a small company, a single contributor might handle everything end-to-end: designing, coding, testing, and deploying. But as the project grows, the team needs to expand. One person might focus on backend, another on frontend, another on testing. To keep everything aligned, a tech lead or VP of engineering steps in, coordinating the workload, assigning tasks, and integrating results. The difference between a lone contributor and a coordinated team maps directly onto the difference between a single agent and a multi-agent system (MAS). Just as a tech lead coordinates multiple contributors, a supervisor agent can orchestrate multiple specialized agents to collaborate on solving a complex goal.

It is also important to recognize that neither humans nor agents operate with complete autonomy. Developers are guided by coding standards, project requirements, and organizational processes, and their freedom is bounded by these structures. Agents are similarly constrained by their workflows, operating only within the scope of the tools, permissions, and safeguards they are provided. Far from being a weakness, this lack of full autonomy is what makes both systems viable. For humans, structure ensures code quality, maintainability, and client satisfaction. For agents, constraints ensure safety, reliability, and alignment with the goals set by their developers and users.

This parallel highlights why extending pure LLMs with tools makes sense. A language model on its own is like a developer cut off from resources like StackOverflow, IDEs, or GitHub, capable of reasoning but unable to gather new data, verify results, or improve their task output without outside help. Just as software teams scale their capabilities by adding better tools, resources, and leadership, LLMs extend their capability when embedded in agentic workflows and even multiple specialized agents within a MAS. Without these upgrades, both remain confined to the limits of their initial training and quickly become overwhelmed when faced with complex tasks. With tools, feedback, and a divide and conquer approach, LLMs can iteratively refine, adapt, and solve complex tasks that would otherwise exceed their standalone capacity.

I assume in this book that you have at least a fundamental knowledge of LLMs. Maybe you've read the book [Hands-On Large Language Models](#), or

a similar work. You don't need to know how to deploy LLMs to make your agents work, since I'll guide you through those steps in later chapters. However, I expect you have solid coding skills in Python, meaning you know what classes and functions are, and that you are familiar with core ML and deep learning concepts such as neural networks, training loops, and backpropagation. A working knowledge of linear algebra and calculus is also helpful, since they underpin much of modern deep learning. It is also useful if you understand how transformers process sequences and have at least a basic sense of how embeddings work or vector stores support retrieval.

If you haven't touched these topics in a while, DON'T PANIC! Just like The Hitchhiker's Guide to the Galaxy advises not to panic, I'll say the same here in my guide. That said, every concept will be grounded in code, so your knowledge becomes usable again without requiring you to dust off old textbooks. And no, before you ask, the answer to everything about AI agents is not 42, but I promise you'll get your answers throughout this book.

CODE FOR THE BOOK

All code examples from this book are available in the [GitHub accompanying repository](#). The repository is organized by chapter, so you can easily find the code that belongs to each section.

If you would like a closer look at transformers across different domains and how embeddings are used for text and other modalities, my book [Transformers: The Definitive Guide](#) can serve as a complementary source. It also explains the fundamentals of reinforcement learning, which will help you build a stronger foundation for understanding the advanced agent concepts introduced later in this book.

Therefore, this book is written for readers who want to go beyond curiosity, which is why understanding these fundamentals is important. This includes data scientists, software and ML engineers who are building real systems in production, and technical leaders who need to understand the trade-offs, costs, and guardrails involved. This book is not about explaining LLMs from scratch, nor is it about abstract promises, or thought experiments. It is about the practical realities of building and running AI agents that can perform and be useful outside a PowerPoint presentation or a lab demo.

This chapter explains how LLMs evolve from static prompts to dynamic agentic systems, why tool use is essential, and what being stateful and autonomous truly means in the design of modern AI agents.

Finite and Hierarchical State Machines: The Base Paradigm of Agents

[LangGraph](#), [CrewAI](#), and similar frameworks build on the concept of state machines. At a high level, a *state machine* is a computational model that exists in one of a finite number of states at any given time. It transitions between states in response to specific events or inputs, with the next state determined by the current state and the input.

This section is meant to help you build a mental model of agents by showing how state machines form the foundation of modern AI agent frameworks. In simple terms, think of it as structured transitions, a pattern long used in compilers, GUIs, embedded systems, and networking protocols, now applied to reasoning and tool use in LLM-based agents.

My intention in grounding your understanding first in finite state machines (FSMs) and hierarchical state machines (HSMs) is more than a loose analogy. I want to give you a blueprint for building LLM agents that are robust, reliable, and adaptable. It will empower you to not only understand the concepts but to apply them effectively as you learn about more complex topics later in the book. However, if you haven't worked with state machines recently, don't worry. You'll see how the patterns translate directly in the code examples.

Finite State Machines

A *finite state machine* has a small vocabulary in which it can act upon and modes your system can be in, and the allowed moves between them. Each move is triggered by an event, optionally guarded by a predicate, and may perform actions that update a state. Start and end are distinguished states with special meaning. The core vocabulary is:

State

A compact snapshot that captures what the system knows at a given moment. For an agent, this might be the message list, routing hints, or progress markers.

Event

Something that happens since the last decision, such as a tool being invoked or returning a result.

Guard

A check that decides which transition to take, based on the current state and latest event.

Action

Work performed during a transition, like invoking a tool, appending a message, saving a checkpoint, or requesting human input.

Termination

A condition that signals the process has reached the end.

[Figure 1-1](#) illustrates a finite state machine for tool use. Each transition is defined by an event, a guard, and an action.

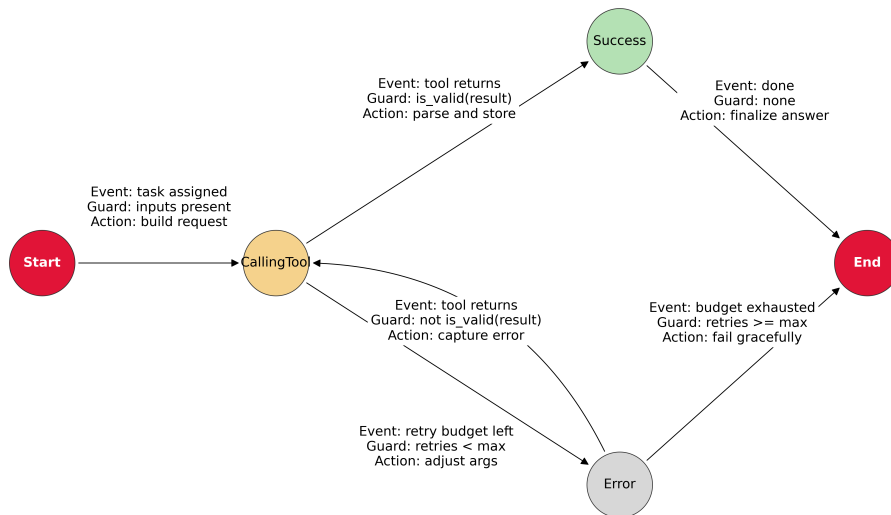


Figure 1-1. High level overview of a finite state machine for tool use.

An FSM is ideal when behavior alternates among a few stable modes and when recoverability matters, since you can checkpoint on every node and resume after a crash from the last completed state.

Thinking in terms of states lets you decompose a complex task into distinct steps. For example, instead of a single prompt for “write a blog post”, you might define states such as topic ideation, outline generation, drafting, editing, and SEO optimization. Each state can link to specific tools your agent will use.

Transitions make you specify the conditions under which one task hands off to the next. For instance, what output from the outline generation state triggers the drafting state. This structure counters unstructured, one-shot prompts that often lead to inconsistent results with LLMs alone.

As soon as you add more modes, such as planning, reflection, approval gates, and retries, an FSM can force duplication or create spaghetti routing. You need to keep control of the edges, since they are decision points: each one encodes the event, guard, and action that drives functionality. This is where hierarchy helps.

Hierarchical State Machines

Hierarchical state machines let states contain other states. Superstates capture shared entry/exit behavior and shared guards. Substates inherit those rules and add their own. History nodes remember which substate you were in when you left, so you can resume exactly there later. These new additions to the state machine are explained below.

Superstate

A state that groups a set of child states and their common policies. Entering the superstate runs shared logic once; exiting it runs shared cleanup once.

Substate

A concrete mode that lives inside a superstate. It has its own edges but inherits the superstate's guards and actions.

History

A “remember where I was” marker. Shallow history resumes at the last active child; deep history resumes inside nested grandchildren. In agent terms, this is a checkpoint for a portion of the graph.

Parallel region

Two or more child regions of a superstate that advance independently. Useful for fanning out tool calls or agent roles and then joining.

[Figure 1-2](#) shows a hierarchical state machine with a **WORKING** superstate. **PLAN**, **ACT**, and **REFLECT** are substates. **H** is a shallow history marker that records the last active substate so execution can resume there if the superstate is re-entered.

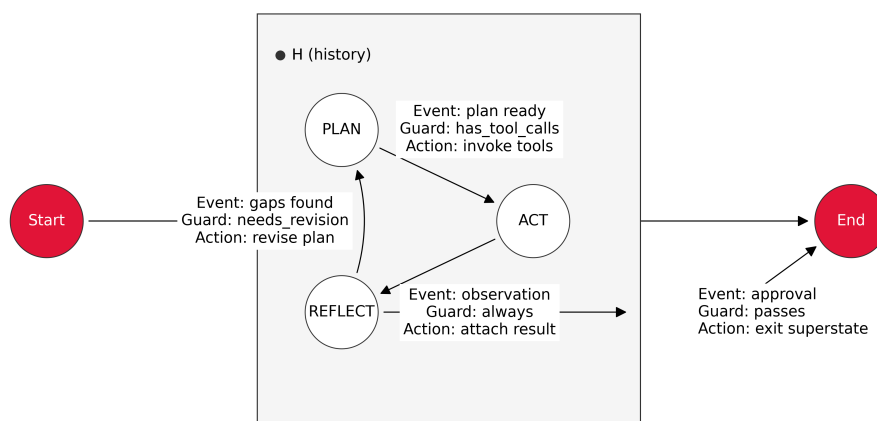


Figure 1-2. Overview of a hierarchical state machine with history marker that records the last active substate.

HSMs are particularly relevant to more advanced MAS. An HSM allows for “states within states”, which reduces duplication and makes policies obvious. For example, you can attach rate limits, safety filters, or circuit breakers to a `WORKING` superstate, so planning, acting, and reflecting all inherit the same safeguards. You define a guard or router once and reuse it across nodes. Consider a research agent. Instead of treating research as a single monolithic step, you can break it into a sub-machine with states such as searching for keywords, gathering URLs, scraping data, and summarizing findings. Each of these states can even map to a dedicated agent with its own capabilities, contributing to the larger team of agents. This nested structure helps manage complexity, keeps policies consistent, and makes the overall system easier to reason about and maintain.

Mapping FSM/HSM to Agent Frameworks

LangGraph already speaks the language of state machines and hierarchy. You implement it with subgraphs and shared keys. The router or decision-making agent that selects the next action (such as calling the next agent or tool) is analogous to the event handler in a classic state machine. It’s the part of the system that takes the current state and the LLM’s output (the event or payload) and determines the next logical step. The direct mapping is shown below.

State schema

The parent graph’s TypedDict defines the shared “bus”. Subgraphs declare which keys they read and write. Overlapping keys are the superstate’s interface.

```
class State(TypedDict):  
    messages: List[BaseMessage] ❶  
    working_last: str | None ❷
```

❶ Shared bus across parent and subgraphs.

❷ Optional shallow history marker.

Nodes and subgraphs

A subgraph is an HSM superstate. Its internal nodes are substates. Entering the subgraph runs its entry node; leaving it runs its exit edge back to the parent.

```
def plan_node(state: State) -> State:  
    ai = AIMessage(content="Plan next step")  
    return {"messages": [ai], "working_last": "plan"}  
  
def act_node(state: State) -> State:
```

```

ai = AIMessage(content="Act on plan")
return {"messages": [ai], "working_last": "act"}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node("PLAN", plan_node)
subgraph_builder.add_node("ACT", act_node)
subgraph_builder.add_edge(START, "PLAN")
subgraph = subgraph_builder.compile() ❶

```

❶ Subgraph is the superstate.

Guards and conditional edges

Parent-level guards enforce global policy (budget, safety). Child-level guards route among PLAN/ACT/REFLECT. Because guards are plain Python, they remain deterministic and auditable.

```

def route_within_working(state: State):
    last = state.get("working_last")
    return "ACT" if last == "plan" else END

subgraph_builder.add_conditional_edges("PLAN",
                                       route_within_working, {"ACT": "ACT", END: END}) ❶

parent = StateGraph(State) ❷
parent.add_node("WORKING", subgraph)

def global_guard(state: State):
    tail = "".join([
        getattr(m, "content", "") for m in state["messages"][-3:]
    ]).lower() ❸
    return END if "stop" in tail else "WORKING"

parent.add_edge(START, "WORKING")
parent.add_conditional_edges("WORKING", global_guard,
                             {"WORKING": "WORKING", END: END})

```

❶ Add conditional edge inside subgraph.

❷ Parent graph with a global guard.

❸ Example policy: end if user said stop.

History and checkpointing

MemorySaver gives you shallow and deep history for free: resuming a thread recreates the subgraph at the last completed node. If you want explicit “return to where I was inside WORKING,” persist a small marker (for example, `state["working_last"] = "reflect"`) and branch on it when re-entering.

```
def reflect_node(state: State) -> State: ❶
    ai = AIMessage(content="Reflect on result")
    return {"messages": [ai], "working_last": "reflect"}

checkpointer = MemorySaver() ❷
app = parent.compile(checkpointer=checkpointer)

cfg = {"configurable": {"thread_id": "session-1"}} ❸
app.invoke({"messages": [HumanMessage(content="start")],
           "working_last": None}, config=cfg)
```

- ❶ Substates set `working_last` on exit.
- ❷ Compile with checkpointing.
- ❸ Run in a thread. `MemorySaver` restores last completed node on resume.

FSMs give you a way to structure tasks into stable, recoverable steps. HSMs extend this by reducing duplication and centralizing policy when workflows grow more complex. Both patterns prepare you for the next step: turning static LLM predictions into dynamic, stateful agents.

Foundations: From Static Models to Dynamic Agents

As you saw in the previous section, adding states and control flows is essential for automation. The same principle applies to agents. To move from static LLM usage to agentic systems, you need explicit state and control flow. State turns one off tool calls into a memory bearing process. Control flow adds guards and transitions that govern when to plan, when to act, and when to stop.

The reason to add this to LLMs is simple: LLMs are static predictors. They generate the next token given a prompt, drawing only on patterns encoded in their training data. This makes them powerful reasoners but leaves them confined to what they already know. They cannot update knowledge, verify claims, or interact with an environment.

Agency emerges when reasoning is coupled with action. *Reasoning* is the internal process of planning or deciding the next step. *Action* extends this by invoking tools, retrieving data, executing code, or calling external services. Together they form an iterative loop: reason, act, receive feedback, and adjust. This loop is the foundation of agentic behavior.

[Table 1-1](#) compares static LLMs to agentic systems. These contrasts show not only why agents are necessary, but also how their design shifts traditional machine learning workflows into dynamic, tool augmented systems.

Table 1-1. Comparison of Static LLMs and Agentic Systems

Static LLM (Traditional Use)	Agentic System
Single-pass token generator	Iterative reasoning–action–feedback system
Confined to training data	Can retrieve, verify, and update knowledge
No memory, stateless	Stateful with short- and long-term memory
Linear prompt–response	Iterative and adaptive workflow
No tool use	Tool-augmented (retrieval, code, APIs)
Fixed interpretation of prompt	Can refine or reinterpret goals dynamically
No external validation	Actively checks, corrects, and improves output

[Figure 1-3](#) illustrates how modules turn a static LLM into a dynamic agent. The user request enters the core, planning and memory shape its reasoning, and tools enable concrete action. The cycle of reasoning, acting, and adapting emerges from the LLM coordination of these elements.

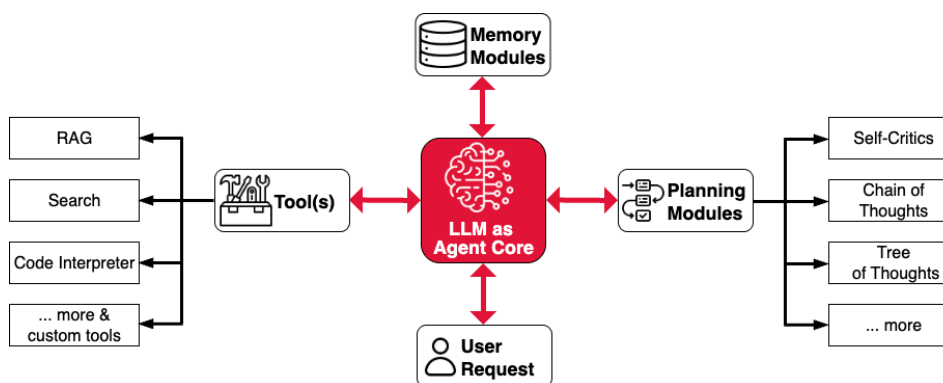


Figure 1-3. Baseline agent architecture: an LLM alone can only predict text, but with planning, memory, and tools it becomes a usable agentic system.

Planning modules guide the agent’s reasoning, ranging from simple chain-of-thought ¹ traces to more advanced approaches such as trees of thought ² or self-critique. Memory modules provide continuity, allowing the agent to recall past steps, reuse prior knowledge, or persist information across sessions. Tools connect the agent to its environment, whether through search, retrieval, code execution, or custom APIs.

Moreover, a key difference between traditional LLM usage and agentic systems is whether the workflow is stateless or stateful. A *stateless call* treats the model like a black box: it takes a prompt, predicts the next to-

kens, and returns a result. Nothing is remembered, no actions are taken, and no feedback loop exists. [Example 1-1](#) illustrates a stateless LLM call.

Example 1-1. Stateless LLM call with LangChain

```
llm = ChatOpenAI(model="gpt-5-mini")
response = llm.invoke("What are AI agents?")
print(response.content)
```

In addition, the moment you need facts from outside training data, or you want the model to act (search, compute), you introduce tools.

In the following code ([Example 1-2](#) - [Example 1-4](#)) is a minimal stateless tool-using run: first you bind two tools to the LLM, and then run a tiny loop that executes any tool calls the model needs to fulfill its task. First, you need to define your tools. [Example 1-2](#) shows an implementation of tools.

Example 1-2. Defining tools for a stateless run

```
@tool("internet_search") ❶
def internet_search(query: str) -> str:
    """Search Google via SerpAPI for up to date information.""" ❷
    serp_api_key = os.environ["SERPAPI_API_KEY"]
    params = {"engine": "google", "gl": "us", "hl": "en"}
    search = SerpAPIWrapper(params=params, serpapi_api_key=serp_api_key)
    return search.run(query)

@tool("calculator")
def calculator(expression: str) -> str:
    """Evaluate a single line mathematical expression with numexpr."""
    local_dict = {"pi": math.pi, "e": math.e}
    out = numexpr.evaluate( ❸
        expression.strip(),
        global_dict={},
        local_dict=local_dict,
    )
    return str(out)

tools = [internet_search, calculator]
tool_map: Dict[str, Any] = {t.name: t for t in tools}
```

- ❶ LangChain's `@tool` decorator uses the function name, typed signature, and docstring to build the schema the model sees.
- ❷ The docstring is mandatory: if it is missing, LangChain will raise an error when constructing the tool schema or binding tools to the model.

- 3 Constrain evaluation: deny globals and expose only the constants you want. This keeps the calculator deterministic and safe.

With LangChain's `@tool` decorator, the function signature and docstring are used to build the schema the model sees. You need to keep these precise, consistent and as explanatory as possible for the task-to-be-done efficiently. The docstring acts as the tool description and is a hard requirement from LangChain, if it's missing, you get an error. In addition, if the docstring is vague the model is more likely to produce invalid calls.

Now, to give the LLM access to the tool, you just bind the tools to the LLM as shown in [Example 1-3](#).

Example 1-3. Bind tools to LLM

```
llm = ChatOpenAI(model="gpt-4o", temperature=0,
                  max_tokens=800).bind_tools(tools, tool_choice="auto") ❶
```

- ❶ Bind tools to the model. `tool_choice="auto"` lets the model decide whether and which tool to call.

MODERN LLMS ARE BUILT FOR AGENTIC TASKS

Modern LLMs are not only capable of calling tools, they are increasingly trained and optimized specifically for agentic tasks. For example, models such as [Kimi K2](#) and [Llama 4](#) are explicitly tuned for coding, tool use, and powering agentic systems. These models go beyond answering questions in a chat window. They are instruction tuned to act, invoking APIs, running code, or retrieving data as part of their core design.

With tools bound, you can run a minimal stateless tool loop. The code in [Example 1-4](#) executes any tool calls the model requests, feeds observations back, and forces a short wrap up to ensure the final answer is returned even if the last step ended on a tool observation.

Example 1-4. Stateless single run with tool loop

```
def run_once(prompt: str, max_steps: int = 8) -> str:
    messages: List[HumanMessage | AIMessage | ToolMessage] = [
        HumanMessage(content=prompt)] ❶
    last_ai: AIMessage | None = None

    for _ in range(max_steps): ❷
        ai: AIMessage = llm.invoke(messages) ❸
        messages.append(ai)
        last_ai = ai

    calls = getattr(ai, "tool_calls", None) or [] ❹
```

```

        if not calls:
            return messages[-1].content

    for call in calls:
        name = call["name"]
        args = call.get("args", {}) or {}
        result = tool_map[name].invoke(args)
        messages.append(ToolMessage(
            content=str(result),
            name=name,
            tool_call_id=call.get("id") ❸
        ))

    messages.append(HumanMessage(content="""
Finish now. Give a short final answer in this exact format:

Current temperature:
Square of current temperature:
""".strip())) ❹
    final_ai: AIMessage = llm.invoke(messages)
    return final_ai.content

print(run_once("""Two step task.

Step 1: Use internet_search to get the current air temperature in New York (
today. Show the exact query you used, the top source title and snippet, and
extract a numeric temperature in Celsius. Return this temperature as feedback
for Step 2.

Step 2: Using the Celsius value from Step 1, compute its square with calcula
Show the exact expression you used and the numeric result.

Important: Give a short final answer in this format:
Current temperature:
Square of current temperature: """))

```

- ❶ Local short term memory for this run only. This makes the workflow stateless across runs.
- ❷ A simple step bound protects against runaway loops: LLMs cycling with no termination.
- ❸ One model step. The assistant may choose to call tools.
- ❹ Read structured tool calls.
- ❺ Attach the observation with the matching tool_call_id so the model can correlate results to calls.
- ❻ Tool cycles can end on an observation. This final instruction guarantees a closing assistant message in your requested format.

Giving the LLM a State

The first step in turning an LLM into an agent is to make it stateful. Give your LLM a state that records where it's in the workflow. Based on input and logic, it transitions to the next state, and each state has an associated function or behavior. In LangGraph, this can be any Python type, but is typically a `TypedDict` or `Pydantic BaseModel`. The following outlines LangGraph's key concepts to build stateful, adaptable AI agents.

State

A shared data structure that represents the current snapshot of an application. It can be any Python type, but is typically a `TypedDict` or `Pydantic BaseModel`.

Nodes

Functions that encode the agent's logic. They take the current state, perform computation or side effects, and return an updated state.

Edges

Rules that select the next node based on the current state. They can be conditional branches or fixed transitions and should also detect when a finish condition is met.

Command

An object that combines control flow and state updates to support multi-actor communication. A node can both update the state and choose the next node by returning a `Command`.

So far, the tools you created earlier have only been used in *stateless loops*. Each run was independent: the model could call tools, get answers, and wrap up, but as soon as the run ended, all context was lost. Now, if you want to combine LangGraph's concepts to build stateful agents, you need to embed the tools into a stateful workflow. Instead of discarding messages after each turn, you keep them in a structured state. This makes the system agentic:

- The LLM node reasons about the next step.
- The tool node executes external actions.
- Edges route control flow based on whether tools were requested.
- Memory checkpoints persist the conversation, so the agent can reuse earlier results in later turns.

This transforms tool use from being a one-off helper into a continuous reasoning-acting-adapting cycle. In the next code listings ([Example 1-5](#) – [Example 1-13](#)), you'll build such a stateful agent with LangGraph, starting

from the same tools and LLM binding introduced in [Example 1-3](#) - [Example 1-4](#).

[Example 1-5](#) shows how a state object can be implemented in LangGraph to carry the conversation and coordinate control flow. You can reuse the tools and the bound llm from [Example 1-3](#) and add only what is new: a typed state, an LLM node that appends assistant messages, a tool node that executes calls, and a router that decides whether to continue or stop based on tool calls.

Example 1-5. Build a minimal LangGraph with state, nodes, and routing

```
class AgentState(TypedDict):  
    messages: Annotated[List[BaseMessage], add_messages] ❶  
  
def llm_node(state: AgentState) -> AgentState: ❷  
    ai = llm.invoke(state["messages"])  
    return {"messages": [ai]}  
  
tool_node = ToolNode(tools=tools) ❸  
  
graph = StateGraph(AgentState) ❹  
graph.add_node("llm", llm_node)  
graph.add_node("tools", tool_node)  
graph.add_edge(START, "llm")  
  
def route(state: AgentState): ❺  
    last = state["messages"][-1]  
    calls = getattr(last, "tool_calls", None) or []  
    return "tools" if calls else END  
  
graph.add_conditional_edges("llm", route, {"tools": "tools", END: END})  
graph.add_edge("tools", "llm")
```

- ❶ The state carries the conversation as a message list.
`add_messages` handles safe merging across node updates.
- ❷ Core reasoning node. It takes the current state and returns a new assistant message.
- ❸ Prebuilt node to execute tool calls emitted by the assistant.
- ❹ Graph container for nodes and edges.
- ❺ Router. If the last assistant turn requested tools, go to tools, otherwise finish.

With the graph defined, you need to make it runnable and persistent.

[Example 1-6](#) compiles the graph with an in-memory checkpointer and as-

signs a thread id. The checkpointer restores prior messages on each turn so the agent can reuse earlier results. The thread id separates branches, so you can run parallel conversations without interference.

Example 1-6. Compile with in-memory checkpoints and configure a thread

```
checkpointer = MemorySaver() ❶  
app = graph.compile(checkpointer=checkpointer) ❷  
  
cfg = {"configurable": {"thread_id": "nyc-weather-session"}} ❸
```

- ❶ Lightweight, per-thread checkpointing that persists state across turns.
- ❷ Produces a runnable app that knows how to save and load state.
- ❸ A thread id identifies one conversation branch. New ids create new branches.

Once you have a runnable app and checkpoints in place, the next step is to make its execution observable. You want to see what the graph is doing, which messages are flowing, and how tool outputs attach. [Example 1-7](#) introduces a compact helper that formats messages into a single readable line, making it easier to follow traces without overwhelming logs.

Example 1-7. Trace execution and inspect state and memory

```
def _short(msg: BaseMessage, max_len: int = 140) -> str:  
    """Compact one-line view of a message."""  
    role = type(msg).__name__.replace("Message", "").lower()  
    content = getattr(msg, "content", "")  
    if isinstance(content, list):  
        # some tool outputs can be list payloads  
        try:  
            content = json.dumps(content)  
        except Exception:  
            content = str(content)  
    text = str(content).replace("\n", " ").strip()  
    if len(text) > max_len:  
        text = text[: max_len - 3] + "..."  
    if hasattr(msg, "tool_calls") and getattr(msg, "tool_calls"): ❶  
        tnames = [tc.get("name", "tool") for tc in msg.tool_calls]  
        return f"{role}: tool_calls -> {tnames}"  
    if isinstance(msg, ToolMessage):  
        return f"{role}({msg.name}): {text}"  
    return f"{role}: {text}"
```

- ❶ Include tool name or function call info when available.

[Example 1-8](#) prints a concise view of messages and routing hints.

Together they let you verify that tool outputs are attached correctly and that state transitions behave as intended.

Example 1-8. Trace execution and inspect state and memory

```
def print_state_snapshot(app, config, title: str):
    """Print current graph state and memory for a given thread."""
    snap = app.get_state(config) ❶
    values = snap.values or {}
    msgs: List[BaseMessage] = values.get("messages", [])
    print(f"\n=== {title} | state snapshot ===")
    print(f"messages: {len(msgs)} total")
    for i, m in enumerate(msgs[-5:], start=max(0, len(msgs)-5) + 1): ❷
        print(f"  {i:>3}: {_short(m)}")

    nxt = getattr(snap, "next", None) ❸
    tasks = getattr(snap, "tasks", None)
    if nxt:
        print(f"next nodes: {list(nxt)}")
    if tasks:
        print(f"queued tasks: {tasks}")
    print("memory: in-memory checkpoint present for this thread")
```

- ❶ Read the latest state for a thread.
- ❷ Print a concise tail of messages to keep logs readable.
- ❸ Show routing info and queued tasks if present.

[Example 1-9](#) streams node updates, so you can see each step the graph takes.

Example 1-9. Trace execution and inspect state and memory

```
def run_with_tracing(app, input_state: AgentState, config, title: str):
    """Run the graph while printing per-node updates and final memory."""
    print(f"\n=== {title} | execution trace ===")
    final = None
    for event in app.stream(input_state, config=config, stream_mode="updates"):
        for node, upd in event.items():
            keys = list(upd.keys()) ❷
            print(f"[enter {node}] updated: {keys}")
            # if messages updated, print the last one briefly
            msgs = upd.get("messages") or []
            if msgs:
                print(f"  {_short(msgs[-1])}")
            print(f"[leave {node}]")
            final = upd
    print_state_snapshot(app, config, title=f"{title} | after run") ❸
    snap = app.get_state(config)
```

- ❶ Stream node updates to see the exact flow through the graph.
- ❷ `upd` is a dict like `{"messages": [<new msg>]}` or tool results.
- ❸ Show final assistant message from `app.get_state`.

[Example 1-10](#) asks for the current air temperature in New York City in Celsius. The trace shows the graph entering the LLM node, initiating a tool call if needed, and returning a final assistant message. The state snapshot confirms that the result is saved in memory for this thread.

Example 1-10. Turn 1: get the current NYC air temperature in Celsius

```
turn1_answer = run_with_tracing(
    app,
    {"messages": [HumanMessage(content="Get the current air temperature in New York City in Celsius.")]},
    config={**cfg, "recursion_limit": 20},
    title="TURN 1",
)
print("\nTURN 1 (final assistant):\n", turn1_answer)
```

This results in this model output:

```
TURN 1 (final assistant):
The current air temperature in New York City is 18°C.
```

You continue in the same thread. [Example 1-11](#) asks to square the previously retrieved temperature. Because the thread id is unchanged, the app restores the earlier messages, allowing the assistant to use the prior result without you repeating it in the prompt.

Example 1-11. Turn 2: square that temperature in the same thread

```
turn2_answer = run_with_tracing(
    app,
    {"messages": [HumanMessage(content="Now compute the square of that temperature.")]},
    config={**cfg, "recursion_limit": 20},
    title="TURN 2",
)
print("\nTURN 2 (final assistant):\n", turn2_answer)
```

Now the square will be computed and the model returns:

```
TURN 2 (final assistant):  
The square of the temperature, 18°C, is 324.
```

Agents often need to explore alternatives without losing progress.

[Example 1-12](#) starts a separate branch by using a different thread id and requests a different follow-up. This creates an isolated conversation that can diverge from the main path while preserving the main thread's memory intact.

Example 1-12. Branch a parallel thread for a different follow-up

```
branch_answer = run_with_tracing(  
    app,  
    {"messages": [HumanMessage(content="Instead of squaring, convert it to  
Fahrenheit and report both.")]},  
    config=cfg_branch,  
    title="BRANCH",  
)  
print("\nBRANCH (final assistant):\n", branch_answer)
```

Finally, you can inspect what each branch remembers. [Example 1-13](#) prints compact snapshots for the main thread and the branch. Comparing them side by side confirms that state is persisted within a thread and isolated across threads, which is the foundation for reliable multi-step and multi-branch agent workflows.

Example 1-13. View snapshots for both threads

```
print_state_snapshot(app, cfg, title="MAIN THREAD memory view")  
print_state_snapshot(app, cfg_branch, title="BRANCH THREAD memory view")
```

With this small implementation, you now see exactly what the graph remembers, how it routed, and what the latest assistant state is for each branch.

Deus Ex Machina: What is Possibly Right Now?

Once you view agents as HSMs with states, the autonomy spectrum becomes a matter of how much choice you place inside a superstate. Routers are one-edge FSMs with a single guard. Tool-calling agents are HSMs with a WORKING superstate that loops among PLAN, ACT, and REFLECT. MAS compose multiple subgraphs under a supervisor superstate, sometimes with parallel regions. The system still runs inside explic-

it states and guards, perceived “autonomy” is just a richer choice inside well-bounded regions.

In ancient Greek theater, a *deus ex machina* was the sudden appearance of a god to resolve an unsolvable conflict or problem. It is tempting to imagine AI agents in the same way: a machine that suddenly acts with full autonomy and intelligence solving all your problems. But that’s not what’s possible today, agents are engineered systems. Their strength comes not from hidden magic, but from carefully designed workflows, well-defined tools, and the boundaries that keep them safe and usable. That said, agents are still constrained by the tools you give them, by safeguards you enforce, and by the workflows you allow them to operate within. These constraints are what make agents viable today: safe to deploy, predictable in behavior, and aligned with their intended goals.

What current systems demonstrate is a form of orchestrated autonomy. As you saw in the LangGraph examples earlier ([Example 1-5](#)), agents can branch conditionally, call tools, or decide whether to continue a workflow. This already looks very different from a static prompt–response call, but it is still bounded. The orchestration logic is fixed; within it, the LLM can make choices, but it cannot redefine its architecture. [Table 1-2](#) gives a short overview what agents can do so far.

Table 1-2. Current autonomy in agents: what’s possible vs. what remains out of reach

What agents can do	What agents can’t do yet
Route between predefined paths (e.g., router node)	Redesign or rewire their own control graph
Select which tools or sub-agents to call	Generate and validate new graph topologies dynamically
Decide if more steps are needed before completion	Continuously assess tool effectiveness or create new tools autonomously
Revise their own prompt or toolset within bounds	Guarantee alignment and coherence when shifting strategies mid-execution
Write and run small pieces of code to determine next steps	Operate as fully self-directed systems independent of orchestration frameworks

The agent adapts at runtime, but only within the framework you design. It helps to see this as a spectrum. At one end are routers that make a single decision from a fixed set of options. In the middle are tool-calling agents that support multi-step reasoning, reflection, and memory. At the far end are MAS, where specialized agents collaborate under a supervisor, distributing decision-making across roles. [Figure 1-4](#) shows the spectrum of autonomy, from simple routing decisions to the idea of fully self-directed execution. In practice, nearly all real-world systems fall into the middle: they are agentic systems, not truly autonomous agents. Their au-

tonomy is orchestrated rather than self-evolving, decisions happen within a framework of predefined states, transitions, and toolsets, even if the agent can adaptively choose among them at runtime.

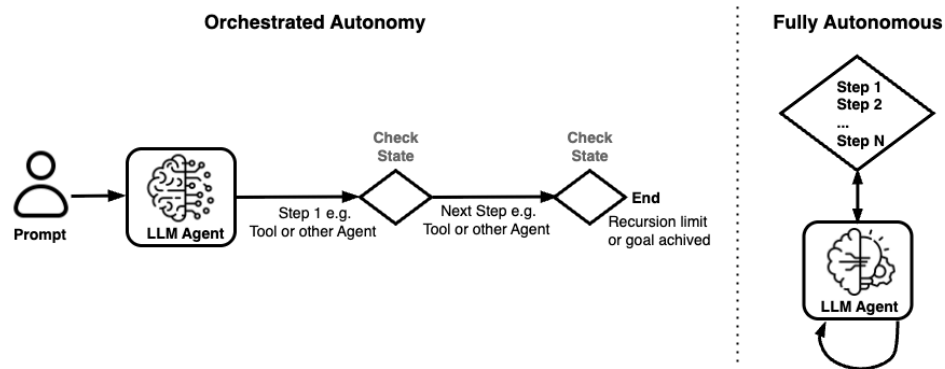


Figure 1-4. In orchestrated autonomy, decision points are predefined in the workflow. In full autonomy, the LLM could generate new tools, actions, or decision paths to reach its goal.

Agents can simulate flexibility within frameworks such as LangGraph using reflection, planning, and structured outputs. However, we do not yet have systems that can freely redefine their own architecture or reason about their goals in a robust, and general way.

The AI Scientist v2³ is an example of what orchestrated autonomy already makes possible. Built on a multi-agent workflow that integrates tree search, experiment management, and vision-language feedback, it can generate research hypotheses, write code to test them, run experiments in parallel while debugging and refining, visualize results, and ultimately draft a scientific manuscript. In 2025 the system submitted three papers to an [ICLR](#) workshop, one of which passed peer review and was formally accepted, the first known case of a fully AI generated scientific paper hitting that bar.

Yet the achievement came with clear limitations. The accepted paper was at workshop level, not a main conference track. From the three submissions, two were rejected, and even the accepted work showed methodological gaps, unclear justifications, shallow analysis, and dataset limitations. Although reviewers praised its clarity and rigor, they stressed that it lacked the depth and innovation required for top tier publication. The system also stumbled on well known LLM pitfalls, including citation errors and misleading figure descriptions. So, while impressive, that the system can refine and debug autonomously, it doesn't generate fundamentally new scientific paradigms autonomously yet.

This case shows both sides of current autonomy. Within the boundaries of engineered orchestration, agentic systems can already close the loop from idea to experiment to code and even peer reviewed publication. Beyond those boundaries, toward consistently producing groundbreaking science or operating with unconstrained creativity, today's agents remain limited and human oversight remains important.

As you now understand, today's agents are not deus ex machina solutions but carefully orchestrated systems that extend LLMs with structure, tools, and feedback loops. Their autonomy is bounded by design, which is precisely what makes them practical, safe, and deployable. While still limited, they are already impressive and capable of reasoning, acting, and adapting in ways that go far beyond static LLMs.

Conclusion

In this chapter, you learned how LLMs evolve from static predictors into dynamic agents through the addition of state, control flow, and tool use. By embedding reasoning in a loop of action and feedback, LLMs gain the ability to adapt to results and extend their capabilities far beyond what is possible with prompt-response interactions alone.

Finite and hierarchical state machines provided the conceptual foundation for agent design. FSMs let you structure workflows into recoverable steps, while HSMs reduced duplication and centralized policies for complex, multi-step tasks. These abstractions map directly to modern frameworks such as LangGraph, where state, nodes, and routing rules define how agents reason and act over time.

You now can also place autonomy in a better context. You understand that today’s agents operate within well-bounded workflows, choosing among predefined paths, invoking tools, and reflecting on results, don’t redesign their own architecture. This orchestrated autonomy is not a weakness but a strength, since constraints make agents safe, predictable, and practical to deploy. The example of the AI Scientist v2 showed both the promise and the limitations of this approach, demonstrating how agents can already generate and test hypotheses but still require oversight to ensure rigor and depth.

With this foundation in place, you are ready to move into the next chapter, where you’ll explore the architectures and patterns that bring planning, reactivity, reflection, and human–agent interaction to life.

- 1** Jason Wei et al. [“Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.”](#) (2023).
- 2** Shunyu Yao et al. [“Tree of Thoughts: Deliberate Problem Solving with Large Language Models.”](#) (2023).
- 3** Yutaro Yamada et al. [“The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search.”](#) (2025).